

Multiple Choice

1. **Answer:** A

Options: A) $a < c$; B) $a < b$; C) $a > b$; D) $a == b$

Note: The condition " $a < c$ " is sufficient to guarantee that the entire expression evaluates to true because if a is less than c , the first part of the disjunction ($a < c$) is satisfied, making the whole expression true regardless of the other conditions.

2. **Answer:** C

Options: A) 4; B) 1; C) 10; D) 8

Note: The correct answer is 10 because the sum function in Python 3 calculates the total of all elements in the list `l`, which are 1, 2, 3, and 4, adding them together to yield 10.

3. **Answer:** D

Options: A) 1; B) 2; C) 3; D) 4

Note: The correct answer is 4 because the `max()` function in Python returns the largest value from the list, and in the list `[1, 2, 3, 4]`, the largest number is 4.

4. **Answer:** A

Options: A) $a[i] == \text{max}$; B) $a[i] != \text{max}$; C) $a[i] < \text{max} \parallel a[i] > \text{max}$; D) FALSE

Note: The expression `!(max != a[i])` simplifies to `max == a[i]`, making the entire expression equivalent to `a[i] == max || a[i] == max`, which further simplifies to just `a[i] == max`.

5. **Answer:** B

Options: A) `a`; B) `Chemistry`; C) 0; D) 1

Note: The correct answer is "Chemistry" because negative indexing in Python allows access to list elements starting from the end, so `[-3]` refers to the third element from the end of the list, which is 'Chemistry'.

True / False

6. **Answer:** True

Options: A) True; B) False

Note: In Python, the slicing syntax `b[::2]` retrieves every second element from the list starting from the first element, which results in the list `[11, 15, 19]` from the original list `[11, 13, 15, 17, 19, 21]`.

7. **Answer:** False

Options: A) True; B) False

Note: The statement is false because the function `min([1, 2, 3, 4])` returns the smallest value in the list, which is 1, not the second smallest value, which is 2.

8. **Answer:** False

Options: A) True; B) False

Note: The function `choice(seq)` in Python 3 is used to select a random element from a non-empty sequence and does not set or alter the starting value for generating random numbers.

9. **Answer:** True

Options: A) True; B) False

Note: In Python 3, the `'+'` operator concatenates two strings, so adding `'a'` and `'ab'` combines them into the single string `'aab'`.

10. **Answer:** True

Options: A) True; B) False

Note: In Python, tuples are zero-indexed, meaning that the first element can be accessed using the index 0, which for the tuple `('abcd', 786, 2.23, 'john', 70.2)` corresponds to `'abcd'`.

Long Form Response

11. **Answer:** Failing to check for a zero value in the denominator when calculating the average of N numbers in an algorithm can lead to a runtime error, specifically a division by zero error. This occurs because dividing by zero is mathematically undefined, and when the algorithm attempts this operation, it will crash or produce an unexpected result. Such errors typically become apparent during runtime, when the program is executing and tries to perform the calculation. To prevent this, programmers should include a check to ensure that the denominator is not zero before performing the division, ensuring the algorithm runs smoothly and produces correct outputs.

Note: correct because failing to check for a zero value in the denominator can cause a division by zero error during runtime, leading to program crashes or unexpected results.

12. **Answer:** When the Python 3 `set` function is applied to the list `l = [1, 2, 2, 3, 4]`, it processes the elements of the list and removes any duplicates, resulting in the output `{1, 2, 3, 4}`. This happens because sets are designed to contain only unique elements, meaning that each value can only appear once. The significance of using sets in programming lies in their ability to simplify tasks that involve uniqueness, such as removing duplicates from a list or checking membership efficiently. By using sets, programmers can write cleaner and more efficient code when dealing with collections of items.

Note: correct because it accurately describes how the `set` function removes duplicates from the list, resulting in a collection of unique elements, and explains the importance of sets in programming for tasks involving uniqueness and efficient membership checks.

End of Answer Key